Project Sprint #2

Github link: github.com/zran28320-lgtm/CS449-Solitaire

1. Demonstration

Youtube/Panopto link:
https://umsystem.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=2f2c0e01-692f-46c6-85d8-b410004849b3

	Feature
1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game
5	Automated unit tests

2. Summary of Source Code

Source code file name	Production code or test code?	# lines of code
logic	Production code	143
gui_app	Production code	129
test_logic	test code	118
	Total	390

3. Production Code vs User stories/Acceptance Criteria

User Story ID	User Story
1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game

User Story ID and Name	**AC ID**	**Class Name(s)**	**Method Name(s)**	**Status(complete or not)**	**Notes(optional)**
1.Choose board size and type	1.1	SolitaireLogic, SolitaireGUI	set_board_parameters(), on_new_game_click()	complete	
	1.2	SolitaireLogic, SolitaireGUI	set_board_parameters(), show_error()	complete	
	1.3	SolitaireGUI	__init__()	complete	
	1.4	SolitaireLogic	set_board_parameters()	complete	
	1.5	SolitaireLogic	set_board_parameters(	complete	

			)		
2. Start a new game of the chosen board size and type	2.1	SolitaireLogic	start_new_game(), _generate_board()	complete	
	2.2	SolitaireLogic, SolitaireGUI	start_new_game(), draw_board()	complete	
3. Making a move	3.1	SolitaireLogic	execute_move()	complete	
	3.2	SolitaireLogic	validate_move()	complete	
	3.3	SolitaireLogic	validate_move()	complete	
	3.4	SolitaireLogic	validate_move()	complete	
4. Checking the end of game	4.1 - 4.4	SolitaireLogic	get_game_rating()	complete	
	4.5	SolitaireLogic	has_any_valid_moves()	complete	

4. Tests vs User stories/Acceptance Criteria

User Story ID	User Story
1	Choose board size and type
2	Start a new game of the chosen board size and type
3	Making a move
4	Checking the end of game

4.1 Automated tests directly corresponding to the acceptance criteria of the above user stories

User Story ID and Name	**Acceptance Criterion ID**	**Class Name (s) of the Test Code**	**Method Name(s) of the Test Code**	**Status (Pass/Fail/In Progress)**
1.Choose board size and type	1.1	TestSolitaire	test_ac_1_1_valid_board_size	Pass
	1.2	TestSolitaire	test_ac_1_2_invalid_board_size	Pass
	1.3	It belongs to the pure GUI default value display logic and is generally not tested in unit tests.		
	1.4	TestSolitaire	test_ac_1_4_change_type_hexagon	Pass

	1.5	TestSolitaire	test_ac_1_5_change_type_diamond	Pass
2. Start a new game of the chosen board size and type	2.1	TestSolitaire	test_ac_2_1_initial_board_state	Pass
	2.2	TestSolitaire	test_ac_2_2_reset_ongoing_game	Pass
3. Making a move	3.1	TestSolitaire	test_ac_3_1_valid_jump	
	3.2	TestSolitaire	test_ac_3_2_invalid_occupied_target	Pass
	3.3	TestSolitaire	test_ac_3_3_invalid_empty_adjacent	Pass
	3.4	TestSolitaire	test_ac_3_4_invalid_out_of_bounds	Pass
4. Checking the end of game	4.1	TestSolitaire	test_ac_4_1_rating_outstanding	Pass
	4.2	TestSolitaire	test_ac_4_2_rating_very_good	Pass
	4.3	TestSolitaire	test_ac_4_3_rating_good	Pass
	4.4	TestSolitaire	test_ac_4_4_rating_average	Pass
	4.5	TestSolitaire	test_ac_4_5_continuing_game	Pass

ChatGPT 免费试用 ×

分享

Act as an expert Python software engineer. I am working on a Peg Solitaire game and need to write unit tests using the unittest framework to satisfy my project's acceptance criteria.

Here is my pure game logic class:

```
# logic.py

class SolitaireLogic:
    """
    Handles the pure game logic for Peg Solitaire.
    0: Out of bounds / Invalid
    1: Peg (Marble)
    2: Empty Hole
    """

    def __init__(self):
        self.board_size = 7
        self.board_type = "English"
        self.grid = []
        self.pegs_left = 0
```

ChatGPT 免费试用 ×

分享

```
        elif self.pegs_left == 3:
            return "Good"
        else:
            return "Average"
```

Please write at least two unit test methods for the SolitaireLogic class inside a TestSolitaire test case:

Test 1: Verify a valid move (Acceptance Criterion 3.1). Set up a board, pick a valid peg, and execute a jump over an adjacent peg into an empty hole. Assert that execute_move returns True.

Test 2: Verify an invalid move (Acceptance Criterion 3.4). Assert that trying to jump out of the board boundaries returns False.

Please use the setUp method to initialize the game and call start_new_game() before testing the moves.

ChatGPT 免费试用 ×

分享

Here's a clean, minimal `unittest` test case that hits exactly what your acceptance criteria are asking for. I'll set things up so the board is deterministic and we explicitly create a valid move scenario (since the default board only has one empty center).

Python 运行

```python
import unittest
from logic import SolitaireLogic


class TestSolitaire(unittest.TestCase):

    def setUp(self):
        """Initialize a fresh game before each test."""
        self.game = SolitaireLogic()
        self.game.set_board_parameters("7", "English")
        self.game.start_new_game()

    def test_valid_move_execution(self):
        """
        Test 1 (AC 3.1):
        Verify that a valid move is executed successfully.
        """
        # Manually create a valid move scenario:
        # Place empty hole at (3,3) center already empty
        # Ensure (3,1) and (3,2) are pegs → jump from (3,1) to (3,3)
        self.game.grid[3][1] = 1  # start peg
        self.game.grid[3][2] = 1  # middle peg
        self.game.grid[3][3] = 2  # end empty (already true, just explicit)

        result = self.game.execute_move(3, 1, 3, 3)

        self.assertTrue(result)
        self.assertEqual(self.game.grid[3][1], 2)  # start now empty
        self.assertEqual(self.game.grid[3][2], 2)  # jumped peg removed
        self.assertEqual(self.game.grid[3][3], 1)  # end now has peg
```

ChatGPT 免费试用 ×

分享

Python 运行

```python
        self.assertEqual(self.game.grid[3][1], 2)  # start now empty
        self.assertEqual(self.game.grid[3][2], 2)  # jumped peg removed
        self.assertEqual(self.game.grid[3][3], 1)  # end now has peg

    def test_invalid_move_out_of_bounds(self):
        """
        Test 2 (AC 3.4):
        Verify that a move outside the board boundaries is rejected.
        """
        # Attempt to move outside the board
        result = self.game.execute_move(0, 0, -2, 0)

        self.assertFalse(result)


if __name__ == "__main__":
    unittest.main()
```

A couple of small but important choices I made (so this doesn't bite you later):

- I **manually enforce a valid move setup** instead of relying on the default board, because your generated board only guarantees one empty hole (center), and not necessarily a ready-to-use jump.
- The invalid test goes **straight out of bounds**, which directly targets AC 3.4 instead of accidentally failing for another reason (like wrong cell state).

If you want, I can help you add a third test for `validate_move` itself or something like `has_any_valid_moves()`—those are usually the ones professors sneak extra points into.

The LLM generated generally working tests, but it made two notable logical oversights that I had to correct. First, in test_valid_move_execution (AC 3.1), the LLM asserted the grid state changed but failed to verify that the total peg count tracking variable (self.game.pegs_left) correctly decreased by 1 after a successful jump. Second, in test_invalid_move_out_of_bounds (AC 3.4), the LLM attempted to jump from (0, 0) to (-2, 0). However, (0, 0) is an invalid zone on the English board and does not contain a peg. Thus, the move is rejected primarily because the starting cell is empty, making it a weak test for boundary logic. I corrected this by starting from a valid edge peg at (2, 0) and attempting to jump off the board to (2, -2). I also added the missing assertion for pegs_left.